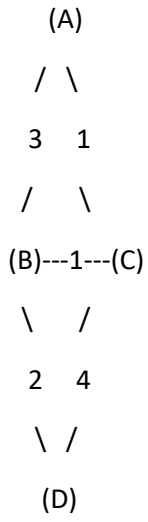
1) Consider the following network topology with 4 routers (A, B, C, D). The link costs are shown below:
   Use the **Distance Vector Algorithm** to calculate the shortest paths from **node A to all other nodes**



Link Costs:

- A–B = 3
- A–C = 1
- B–C = 1
- B–D = 2
- C–D = 4

Step 1: Initialization

Each router knows only the direct link costs. Others are set to ∞.

- A: [A=0, B=3, C=1, D=∞]
- B: [A=3, B=0, C=1, D=2]
- C: [A=1, B=1, C=0, D=4]
- D: [A=∞, B=2, C=4, D=0]

Step 2: First Update (exchange with neighbors)

Router A:

- Via B: A → B → C = 3+1 = 4 (no improvement, since A→C=1).
- Via B: A → B → D = 3+2 = **5** (update D=5).
- Via C: A → C → B = 1+1 = **2** (update B=2, better than 3).
- Via C: A → C → D = 1+4 = 5 (equal to previous).

New A = [0, 2, 1, 5].

Router B:

- Via A: B → A → C = 3+1 = 4 (no improvement, since B→C=1).
- Via A: B → A → D = 3+∞ = ∞ (no update).
- Via C: B → C → A = 1+1 = **2** (update A=2, better than 3).
- Via C: B → C → D = 1+4 = 5 (no improvement, since B→D=2).

New B = [2, 0, 1, 2].

Router C:

- Via A: C → A → B = 1+3 = 4 (no improvement, since C→B=1).
- Via A: C → A → D = 1+∞ = ∞ (no update).
- Via B: C → B → A = 1+3 = 4 (but we already know A=1).
- Via B: C → B → D = 1+2 = **3** (update D=3, better than 4).

New C = [1, 1, 0, 3].

Router D:

- Via B: D → B → A = 2+3 = 5 (update A=5).
- Via B: D → B → C = 2+1 = **3** (update C=3, better than 4).
- Via C: D → C → A = 4+1 = 5 (equal to above).

New D = [5, 2, 3, 0].

Step 3: Convergence

After one more round of updates, the tables stabilize (no further changes).

- **A:** [0, 2, 1, 5]
- **B:** [2, 0, 1, 2]
- **C:** [1, 1, 0, 3]
- **D:** [5, 2, 3, 0]

Final Answer (Shortest Path Costs)

- From A:
  - B = 2 (via C)
  - C = 1 (direct)
  - D = 5 (via B or C)
- From B:
  - A = 2 (via C)
  - C = 1 (direct)
  - D = 2 (direct)
- From C:
  - A = 1 (direct)
  - B = 1 (direct)
  - D = 3 (via B)
- From D:
  - A = 5 (via B or C)
  - B = 2 (direct)
  - C = 3 (via B)

This shows how Distance Vector Routing updates routing tables until they converge.

2.Consider the following network topology with 5 routers (A, B, C, D, E). The link costs are shown below:

Link Costs:

- A–B = 2
- A–C = 5
- B–C = 1
- B–D = 2
- C–D = 3
- D–E = 1

Use the **Distance Vector Algorithm** to calculate the shortest paths from **node A to all other nodes**.

Solution:

Step 1: Initialization (A knows only direct neighbors)

At the beginning, each router knows the cost to its neighbors and infinity (∞) to others.

For A:

- Distance(A, A) = 0

- Distance(A, B) = 2
- Distance(A, C) = 5
- Distance(A, D) = ∞
- Distance(A, E) = ∞

Step 2: First Update (using neighbors' information)

A receives distance vectors from B and C:

From B:

- Distance(B, A) = 2
- Distance(B, C) = 1
- Distance(B, D) = 2
- Distance(B, E) = ∞

So, A can update through B:

- To D via B = Distance(A, B) + Distance(B, D) = 2 + 2 = **4** (better than ∞ → update).
- To C via B = 2 + 1 = **3** (better than direct 5 → update).
- To E via B = 2 + ∞ = ∞ (no update).

From C:

- Distance(C, A) = 5
- Distance(C, B) = 1
- Distance(C, D) = 3
- Distance(C, E) = ∞

So, A can check through C:

- To D via C = 5 + 3 = 8 (but we already found 4 → no update).
- To E via C = ∞ (no update).

Step 3: Second Update (using new distances)

Now A's table:

- A = 0
- B = 2
- C = 3
- D = 4
- E = ∞

Now consider D's information (via B or C):

- Distance(D, B) = 2
- Distance(D, C) = 3
- Distance(D, E) = 1

So, A can compute:

- To E via D = Distance(A, D) + Distance(D, E) = 4 + 1 = **5** (update).

Step 4: Final Distance Vector for A

- Distance(A, A) = 0
- Distance(A, B) = 2
- Distance(A, C) = 3
- Distance(A, D) = 4
- Distance(A, E) = 5

Final Answer:
The shortest path distances from **A** are:

- A → B = 2
- A → C = 3 (via B)
- A → D = 4 (via B)
- A → E = 5 (via D)

In **Computer Networks**, especially in **routing algorithms** like **Distance Vector** or **Link State**, a **link cost** is a **numerical value assigned to a network connection (link) between two routers/nodes**.

Meaning of Link Cost:

- It represents the **"expense"** of sending data over that link.
- Smaller cost = better/cheaper/faster path.
- Larger cost = less preferred path.

How Link Cost is Measured:

Depending on the network, **cost** can mean different things:

1. **Hop Count** → Cost = 1 for every link (used in RIP protocol).
2. **Bandwidth** → High bandwidth = low cost, low bandwidth = high cost.
3. **Delay** → Links with higher delay have higher cost.
4. **Reliability** → More reliable links may be given lower cost.
5. **Administrative value** → Network admin can manually set cost.

Example:

In the network:

```
A ----2---- B
A ----5---- C
```

- Link **A–B** has cost **2** → cheaper than A–C.
- If we want to go from A to C, the algorithm might prefer going **A → B → C** if that total cost is lower than direct **A–C** (which is cost 5).

So in short:
Link cost = numerical weight of a connection that helps routing algorithms choose the best (shortest/cheapest) path.